

passed to server_1 and one to server_2.

```
upstream web_servers {
    server server_1 unixclinic.net:8001 weight=2;
    server server_2 unixclinic.net:8001;
}
```

- The **max_fails** parameter specifies the maximum number of failed connection attempts with a member server within a specified time period. This time period is specified by another parameter **fail_timeout**. The default value of this parameter is 1, and if this is set to 0 then the check is disabled. Setting this to 0 is certainly not recommended and I would advise setting this to at least 2 or 3.
- The **fail_timeout** parameter contains the time duration in seconds. If all connection attempts with a member server for the specified time fail, then this would count as one failure. In our configuration above, the **fail_timeout** value is 30 seconds. This means that if the reverse proxy's connection attempts with one or more of our member servers failed for 30 seconds, then it would count as failed once, and another 30 second failure consecutively will take the 'failed' count to two. After the failed count reaches two, which is what is the value of **max_fails** parameter in our case, the member server is marked as unresponsive for a certain amount of time. The default value for this is 10 seconds. And this timeout is controlled by other directives called **proxy_connect_timeout** and **proxy_read_timeout**. Read the [nginx](#) wiki for more details about this.
- The **down** parameter marks the member server as permanently down. This is typically used with the directive **ip_hash** (discussed below).
- The **backup** parameter is only available in **nginx** version 0.6.7 or later. So if you decide to compile the latest **nginx** version for your use then this could be used. This parameter specifies the member server as a back-up server in case all the other member servers are busy or down.

The Debian Etch has **nginx** version 0.4.13-2, whereas the 'backports' repository has **nginx** version 0.5.35-1. If you have decided to configure the 'backports' repository in your server and then install **nginx**, you can make use of a new feature of the upstream directive, which is available since version 0.5.18. This feature allows you to log a few additional variables via the log module. These variables are:

- **\$upstream_addr**—address of the upstream server that handled the request
- **\$upstream_status**—upstream server status of the answer
- **\$upstream_response_time**—this is recorded in milliseconds. Several answers will be divided by commas and dots.
- **\$upstream_http_\$HEADER**

Finally, to activate the reverse proxy, issue the following commands:

```
# cd /etc/sites-enabled
# ln -s ../sites-available/rev-proxy-lb
# nginx -t
# invoke-rc.d nginx reload
```

After this, visit <http://unixclinic.net/index.php>. Each time you access it, you should notice that the pages are being served according to the weight and from different servers. I have specified **index.php** explicitly because **index.html** has a higher priority on this, and if you remember, we have created **index.html** above, while testing the Web server.

Bottlenecks in this set-up

The websites are redundant and load balanced according to the simple algorithm that **nginx** provides. There are a few bottlenecks in this set-up, which can cause a hurdle in providing a true highly available set-up. The first bottleneck that we can easily see is that we have only one reverse proxy server and in case this goes down, our website too will be down. In order to resolve this we will need to set up another instance of a reverse proxy server as a secondary server, which will take control of the domain(s) being served in case the primary load balancer goes down. In other words, we will need to set up an active-passive clustering between the two **nginx** reverse proxy servers.

We will take up this and some other issues in a subsequent article to see how we can achieve a better redundancy. 

Resources

Nginx and Php5 Fast-cgi

- **nginx** FastCGI example: wiki.codemongers.com/NginxFcgiExample
- **nginx** tips: dominiek.com/articles/2007/5/18/bye-redskin-zdravstvujte-redstar-nginx-tips
- **nginx**, PHP and a PHP FastCGI daemon init script: blog.codefront.net/2007/06/11/nginx-php-and-a-php-fastcgi-daemon-init-script
- How to set up **Ubuntu** with **nginx**/php/mysql/ruby/radiant/mongrel/monit and a simple firewall: johnmuhl.com/notebook/server
- **Nginx** HTTP Server/PHP5 (with FastCGI and xcache) on **Ubuntu**: www.howtoforge.com/nginx_php5_fast_cgi_xcache_ubuntu7.04

Load Balancing

- Round-robin DNS: en.wikipedia.org/wiki/Round_robin_DNS
- How to configure round-robin and load-balancing: www.zytrax.com/books/dns/ch9/r.html
- Load balancing Web applications: www.onjava.com/pub/a/onjava/2001/09/26/load.html

By: Ajitabh Pandey

The author has more than 12 years of diversified IT industry experience in training, support and consulting. You can know more about him at ajitabhpandey.info and shoot him an e-mail at ajitabhpandey@ajitabhpandey.info